

Хабр

Весна новых возможностей

Свежие IT-события уже в Хабре. Каждый день.

Хабр



Чему научиться в этом году?



Sivchenko_translate

12 часов назад

Выделение регистров процессора при помощи генетического алгоритма



16 мин



6.5K

Программирование*, Алгоритмы*, C#*, .NET*, Процессоры

Перевод

Автор оригинала: Kunal S. Pathak

Эксперимент, который многое объясняет

Оригинал этого поста также вошёл в число документов по проектированию платформы .NET: [lsra-heuristic-tuning](#).

Контекст

В динамическом компиляторе RyuJIT реализован алгоритм [линейной схемы распределения регистров](#) (LSRA), в соответствии с которым регистры процессора присваиваются сгенерированному коду. При выборе регистров алгоритм LSRA применяет разнообразные эвристики (если быть точным, их 17), выбирая из всех доступных регистров тот, который лучше всего подходит в настоящий момент. Все регистры можно разделить на две категории. Либо регистр не содержит значения переменной, поэтому может быть выделен для очередной переменной, либо, он уже содержит значение переменной и, следовательно, он занят. Если в процессе присваивания регистров выбран тот, который занят, то значение, которое в нём сейчас содержится, приходится предварительно сохранить в памяти, а потом уже присвоить туда, куда нужно. Такая операция называется «выгрузкой переменной». В алгоритме LSRA компилятора RyuJIT есть (14 видов) эвристики, в соответствии с которыми сначала выбирается один из свободных регистров. Если свободных регистров не окажется, то есть ещё 4 эвристики, согласно которым выбирается один из занятых регистров. Занятый регистр выбирается в зависимости от того, содержимое какого регистра будет дешевле выгрузить.

Мы заметили, что отдавать приоритет при выборке именно свободным регистрам не всегда полезно. Иногда лучше выбрать занятый регистр, а свободные сберечь на будущее для тех опорных точек, которые попадутся на «горячем» пути выполнения кода.

Рассмотрим [сгенерированный код](#), взятый из [dotnet/runtime Issue#8846](#). В данном примере свободные регистры отводятся тем переменным, которые находятся вне цикла for. В ходе присваивания регистров переменным, находящимся внутри цикла, свободных регистров не достаётся, поэтому алгоритму приходится выгружать регистр, занятый в настоящий момент, чтобы сохранить значение переменной, находящейся внутри цикла. Поразительно, но оказалось, что алгоритм выбирает один и тот же регистр для всех переменных, используемых внутри цикла, и раз за разом выгружает значения переменных, находившихся в регистрах ранее. Складывается впечатление, что это происходит именно из-за того, в каком порядке применяются разные эвристики для выбора регистров. Может быть, лучше придерживаться не фиксированного порядка

следования эвристик, а откорректировать этот порядок так, чтобы *иногда* сначала выбирались занятые регистры, а потом шёл выбор из пула доступных свободных регистров. Именно так зародилась идея настроить эвристику выбора регистров именно так, как было описано в [dotnet/runtime Issue# 43318](#). Требовалось поставить эксперимент и проверить, можно ли улучшить порядок выбора регистров, опираясь на иные критерии. В этой статье подробно рассказано, почему для этой цели было решено использовать генетический алгоритм, и чему удалось научиться в ходе этих экспериментов.

Эвристика выбора регистров

Ниже перечислены эвристики выбора регистров, реализованные в компиляторе RyuJIT:

Сокращённое название	Имя	Описание
A	FREE	В настоящее время не присвоен ни одному <i>активному</i> интервалу.
B	CONST_AVAILABLE	Константное значение, которое уже присутствует в регистре.
C	THIS_ASSIGNED	Регистр уже присвоен интервалу, актуальному в настоящий момент.
D	COVERS	Покрывает актуальное время жизни интервала.
E	OWN_PREFERENCE	Набор регистров, предпочтительных для данного интервала.
F	COVERS_RELATED	Набор регистров, предпочтительных для интервала, который касается актуального интервала и покрывает его время жизни.
G	RELATED_PREFERENCE	Набор регистров, предпочтительных для интервала, который касается актуального интервала.
H	CALLER_CALLEE	Набор регистров, сохранённых вызывающей или вызываемой стороной
I	UNASSIGNED	В настоящее время не присвоен ни одному активному или неактивному интервалу.
J	COVERS_FULL	Покрывает время жизни интервала от данного момента и до конца.
K	BEST_FIT	Доступный диапазон ближе всего совпадает с полным диапазоном интервала
L	IS_PREV_REG	Регистр ранее был присвоен актуальному интервалу
M	REG_ORDER	Прерыватель. Просто выбирает первый доступный «свободный» регистр
N	SPILL_COST	Из всех кандидатов стоимость выгрузки для этого будет самой низкой

O	FAR_NEXT_REF	Следующая ссылка на него расположена дальше, чем ссылка на наилучшего кандидата, найденного к текущему моменту.
P	PREV_REG_OPT	Предыдущая ссылка на присвоенный в данный момент интервал была опциональной.
Q	REG_NUM	Прерыватель. Просто выбирает первый доступный «занятый» регистр.

Эвристики от A до M предназначены для выбора одного из свободных регистров, а эвристики от N до Q — для выбора одного из занятых. Ниже на простом примере показано, как эвристика выбора работала ранее. Начинаем со свободных (доступных) кандидатов и, с применением каждой эвристики, сужаем круг этих кандидатов. Всякий раз, когда видим, что у нас есть выбор (нам подходит более одного регистра), продолжаем пробовать эвристики (в вышеуказанном порядке) до тех пор, пока у нас не останется всего одного регистра. Если ни одного подходящего регистра найти не удастся, то с применением эвристики N продолжаем поиск, пока не найдём среди занятых регистров такой, содержимое которого можно выгрузить в стек.

```

registerCandidates = 0; // битовая маска для всех регистров

LinearScan::allocateReg(RefPosition refPosition, Interval* interval)
{
    bool found = false;
    registerCandidates = allFreeCandidates();

    if (!found) {
        found = applyHeuristics(FREE, FREE_Candidates());
    }

    if (!found) {
        found = applyHeuristics(CONST_AVAILABLE_Candidates());
    }

    ...

    if (!found) {
        found = applyHeuristics(REG_ORDER_Candidates());
    }

    // Свободных регистров в доступе не оказалось, поэтому попытаемся выбрать
    // один из занятых регистров
    registerCandidates = allBusyCandidates();
    if (!found) {
        found = applyHeuristics(SPILL_COST_Candidates());
    }

    if (!found) {
        found = applyHeuristics(FAR_NEXT_REF_Candidates());
    }

    ...
}

// Фильтрует регистры-кандидаты и возвращает true только после того, как

```

```
// останется всего один кандидат.
bool applyHeuristics(selected_candidates)
{
    filtered_candidates = registerCandidates & selected_candidates;
    if (filtered_candidates != 0) {
        registerCandidates = filtered_candidates;
        return isSingleRegister(registerCandidates);
    }
    return false;
}
```

Объяснить с  SourceCraft

Если бы мы хотели изменить порядок следования эвристик, то должны были бы обновить вышеприведённый код, так, чтобы переупорядочить применяемые эвристики. Если требуется экспериментировать с разными вариантами порядка эвристик, то выполнять такой рефакторинг для проверки каждой комбинации не представляется возможным. Немного исследовав проблему, подбирая, какой паттерн проектирования подходит для решения такой задачи, мы решили действовать дедовским методом и вынесли код каждой отдельно взятой эвристики в её собственный метод (помечаемый как `__forceinline`, чтобы пропускная способность никак не сказывалась на изменениях, вносимых при рефакторинге). Для вызова любого из этих методов в таком порядке, как мы желаем, можно было бы воспользоваться указателем функции. Наконец, оставалось добавить механизм, который позволял бы пользователю самостоятельно задавать, в каком порядке он хочет попробовать эти методы. Каждой эвристике мы присвоили ровно одну букву(столбец Сокращённое название в таблице выше) и предоставили переменную окружения `COMPlus_JitLsraOrdering` в качестве средства для указания порядка. По умолчанию используется порядок " ABCDEFGHIJKLMNOPQ " (действует сейчас), но, если задать какой-то другой порядок, допустим, " PEHDCGAIJNLKOBFMQ ", то эвристики будут следовать именно так. Например, P соответствует вариант `PREV_REG_OPT`, при котором сначала эвристически проверяются занятые регистры, затем `OWN_PREFERENCE`, `CALLER_CALLEE` и так далее. Как видите, теперь можно сначала применять эвристику проверки занятых регистров, а лишь затем переходить к эвристикам для свободных регистров.

Собрав всё это вместе, получаем после рефакторинга следующий код:

```
typedef void (RegisterSelection::*HeuristicFn)();
HashTable<char, HeuristicFn> ScoreMappingTable = {
    {'A', try_FREE},
    {'B', try_CONST_AVAILABLE},
    ...
    {'Q', try_REG_NUM}
};

LinearScan::allocateReg(RefPostion refPosition, Inteval* interval)
{
    char *ordering = Read_COMPlus_LsraOrdering();
    HeuristicFn fn;
    for (char order in ordering) {
        if (ScoreMappingTable->Lookup(order, &fn)) {
            bool found = (this->*fn)();
            if (found) {
                break;
            }
        }
    }
}
```

```

}

bool LinearScan::try_FREE() {
    ...
    return applyHeuristics();
}
...
bool LinearScan::try_CONST_AVAILABLE() {
    ...
    return applyHeuristics();
}
...
bool LinearScan::try_REG_NUM() {
    ...
    return applyHeuristics();
}
}

```

Объяснить с 

В [dotnet/runtime #52832](#) содержатся все вышеописанные изменения, вступившие в силу после рефакторинга.

Измерение влияния

Теперь, реализовав возможность переупорядочивания эвристик при помощи

`COMPlus_JitLsraOrdering`, мы решили измерить, как эти перемены отражаются на выполнении инструмента [superpmi](#). Инструмент `superpmi` динамически компилирует все методы заданного файла сборки (*.dll или *.exe), не выполняя сгенерированного при этом машинного кода. Имея две версии `clrjit.dll` (двоичный файл RyuJIT), также можно сравнить сгенерированный код и отчитаться о том, сколько методов улучшилось/ухудшилось по показателям `CodeSize` (размер машинного кода), `PerfScore` (задержка при выполнении инструкций/измерение пропускной способности), `InstructionCount` (количество инструкций в наличии) и т.д. Мы выбрали метрики `PerfScore`, поскольку они точно характеризуют, во сколько обходится выгрузка регистров. Если LSRA не удастся оптимально подобрать регистры, то мы увидим несколько инструкций `mov`, загружающих значения в память/сохраняющих их, и именно из-за этого как снизится пропускная способность, так и возрастёт задержка. Соответственно, `PerfScore` станет ниже. Если выгрузка регистров произойдёт внутри цикла, то метрики `PerfScore` учтут и это. Для оценки этого показателя потребуется проверить произведение весов блочных циклов и `PerfScore`. Следовательно, наша цель — уменьшить `PerfScore`, насколько это возможно. Чем ниже будет `PerfScore` — тем лучше код, который мы сгенерировали. В качестве базового уровня для сравнения мы взяли вариант упорядочивания, действующий по умолчанию, после чего решили сравнить его с вариантом, указанным в `COMPlus_JitLsraOrdering`. Можно было задать любую комбинацию последовательностей от A до Q и откорректировать алгоритм LSRA так, чтобы порядок следования эвристик в ней изменился. Но, поскольку эвристик 17, будет **355 687 428 096 000** (факториал 17) возможностей, и перепробовать их все практически невозможно. Нам было просто необходимо найти, как решать эту задачу эффективнее!

Генетический алгоритм

Для решения задач такого рода отлично подходят [генетические алгоритмы](#). Если вы не знаете, что это такое, изложим кратко их суть. Алгоритм начинается с популяции, в которой есть несколько кандидатов, и у каждого кандидата есть заранее определённый показатель приспособленности. Каждый кандидат содержит последовательность генов, причём, количество генов у любого из

кандидатов одинаковое. Алгоритм выбирает пару подходящих кандидатов (родители) и получает от них потомство, причём, в ходе такого размножения геномы родителей претерпевают мутации. Алгоритм вычисляет показатель приспособленности полученных потомков и добавляет их в пул популяции (вместе с их показателями приспособленности). По мере развития популяции в ней становится всё больше кандидатов, чей показатель приспособленности равен исходному показателю приспособленности всей популяции или лучше него. Разумеется, популяция бесконечно расти не может, поэтому наименее приспособленные кандидаты вымирают. Как только на определённом этапе в популяции не остаётся кандидатов с иным показателем приспособленности кроме наивысшего, алгоритм останавливается и выдаёт нам набор наилучших кандидатов.

Такой алгоритм отлично описывает задачу, в которой нужно выбрать наилучший порядок следования эвристик выбора. Мы начинаем с " ABCDEFGHIJKLMNOPQ " (порядок следования эвристик, действующий по умолчанию), и каждую букву в этой комбинации можно трактовать как ген. Генетический алгоритм привносит в геном мутации, в результате которых порядок следования меняется, скажем, на " ABMCDEFGHIKLNJOPQ ". Это значение мы запишем в переменную `COMPlus_JitLsraOrdering`. Затем выполним [superpmi.py](#), чтобы сгенерировать код, и сравним его значение `PerfScore` с тем, которое получается при применении порядка, действующего по умолчанию. Здесь `PerfScore` характеризует приспособленность. Чем ниже показатель этой метрики, тем лучше приспособлен кандидат, что в нашем случае означает: «тем лучше порядок следования эвристик».

Ниже приведён псевдокод того генетического алгоритма, с которым мы экспериментировали, подыскивая оптимальный порядок следования эвристик.

```
// Максимальная численность популяции на поколение
int MaxPopulation = 100;

HashMap<string, float> Community = new HashMap<string, float>();
HashMap<string, float> NextGen = new HashMap<string, float>();

void GeneticAlgorithm() {
    PopulateCommunity();

    do {
        // новое поколение
        NextGen = new HashMap<string, float>();
        candidateCount = 0;

        while(candidateCount++ < MaxPopulation) {
            // Турнирным методом выбираем
            // 2 кандидата из множества "Популяция".
            // https://en.wikipedia.org/wiki/Tournament_selection
            (parent1, parent2) = DoSelection();

            // Гены parent1 и parent2 мутируют, и в результате получаются
            // 2 потомка
            (offspring0, offspring1) = MutateGenes(parent1, parent2)

            // Добавляем потомков в популяцию
            AddNewOffspring(offspring0)
            AddNewOffspring(offspring1)
        }
    }
```

```

        Community = NextGen;

        // Продолжаем процесс в виде цикла, пока в популяции не останутся
        // лишь уникальные кандидаты

    } while (uniqueCandidates);

}

// Заполняем популяцию случайными кандидатами
void PopulateCommunity() {
    candidateCount = 0;
    while(candidateCount < MaxPopulation) {
        newCandidate = GetRandomCombination("ABCDEFGHJKLMNOPQ")
        AddNewOffspring(newCandidate)
    }
}

// Запускаем инструмент superpmi, и с его помощью считываем показатели PerfScore
void ComputeFitness(candidate) {
    perfScore = exec("superpmi.py asmdiffs -base_jit_path default\clrjit.dll -diff_jit_
    return perfScore
}

// Вычисляем показатели приспособленности для обоих потомков и
// добавляем их в сообщество
void AddNewOffspring(candidate) {
    Community[candidate] = ComputeFitness(candidate)

    // выкидываем менее приспособленных кандидатов
    if (Community.Count > MaxPopulation) {
        weakCandidate = CandidateWithHighestPerfScore(Community);
        Community.Remove(weakCandidate)
    }
}

// Выполняем операции, соответствующие скрещиванию и мутации
void MutateGenes(offspring0, offspring1) {
    assert(offspring0.Length == offspring1.Length)

    // скрещивание
    crossOverPoint = random(0, offspring0.Length)
    i = 0
    while (i++ < crossOverPoint) {
        char c = offspring0[i]
        offspring0[i] = offspring1[i]
        offspring1[i] = c
    }

    // мутация
    randomIndex = random(0, offspring0.Length)
    char c = offspring0[randomIndex]
    offspring0[randomIndex] = offspring1[randomIndex]
    offspring1[randomIndex] = c
}

```

```
    return offspring0, offspring1
}
```

Объяснить с 

Имея готовый генетический алгоритм, можно в ходе нескольких экспериментов найти оптимальный порядок следования эвристик.

Эксперименты

При помощи `superpmi` можно динамически компилировать все методы, присутствующие в библиотеках `.NET` и `Microbenchmarks`. Нам требовалось провести этот эксперимент для всех поддерживаемых нами операционных систем и архитектур — `Windows/x64`, `Windows/arm64`, `Linux/x64`, `Linux/arm` и `Linux/arm64`.

Конфигурация

Для проведения экспериментов мы несколько изменили, как `superpmi` собирает `PerfScore` и выдаёт отчётность по этим показателям.

1. `superpmi.exe` была изменена так, чтобы агрегировать **относительную** разницу `PerfScore` в коде, генерируемом по умолчанию и в коде, в котором изменён порядок следования эвристик при `LSRA`. Когда `superpmi.exe` выполняется параллельно (так устроено по умолчанию), это значение отдельно выдаёт в консоль каждый из параллельно работающих процессов.
2. `superpmi.py` была изменена так, чтобы далее агрегировать относительные отличия `PerfScore` у параллельных процессов `superpmi.exe`, после чего выдаёт в качестве отчёта окончательную относительную разницу `PerfScore`.
3. Алгоритм `LSRA` расставляет по всей базе кода множество утверждений. В соответствии с ними, при выборе регистров сначала пробуем найти свободные регистры, а если таковых нет — то переходим к поиску среди занятых. Поскольку мы также хотели понять, как сказывается на работе программы ситуация, в которой предпочитают, в первую очередь, занятые регистры, эти утверждения предварительно требовалось отключить.
4. `superpmi.exe asmdiffs` принимает две версии `clrjit.dll`, которые вы хотите сравнить. Они должны происходить из разных мест. В нашем случае хотелось поэкспериментировать лишь с разным порядком эвристик, передавая разные значения `COMPlus_JitLsraOrdering`. Для этого мы скопировали `clrjit.dll` — `> copy_clrjit.dll` и передавали различные варианты порядка скопированному `copy_clrjit.dll`.

Вот пример вызова `superpmi.py`, при помощи которой генетический алгоритм получает `PerfScore` (балл приспособленности) для каждого экспериментального варианта порядка:

```
python superpmi.py asmdiffs -f benchmarks -base_jit_path clrjit.dll -diff_jit_path copy
```

Объяснить с 

Результат

Ниже показаны порядки следования эвристик, которые алгоритм выдал для различных конфигураций (сценарий/операционная система/архитектура). В столбце PerfScore представлено агрегированное значение относительной разницы PerfScore для всех методов. Мы предпочли работать с относительными значениями разницы, а не с абсолютными, так как не хотели, чтобы показатели доминирующего метода маскировали показатели других, более мелких.

Конфигурация	Порядок	PerfScore
windows-x64 Benchmarks	EHPDGAJCBNKOLFIMQ	-36.540712
windows-x64 Libraries	PEHDCGAIJNLKOBFMQ	-271.749901
windows-x86 Benchmarks	EHDCFPGJBIALNOKMQ	-73.004577
windows-x86 Libraries	APCDEGHNIOFJKLBMQ	-168.335079
Linux-x64 Benchmarks	HGIDJNLCPOBKAEFMQ	-96.966704
Linux-x64 Libraries	HDGAECNIPLBOFKJMQ	-391.835935
Linux-arm64 Libraries	HECDBFGIANLOKJMPQ	-249.900161

Как понятно из этой таблицы, есть значительно более выигрышные варианты упорядочивания, чем используемый по умолчанию " ABCDEFGHIJKLMNOPQ ". Алгоритм может обеспечить значительно более качественный подбор регистров и, следовательно, повысить производительность. Но также наблюдаем, что предложенные генетическим алгоритмом варианты упорядочивания отличаются от конфигурации к конфигурации. Мы хотели найти общий схожий вариант, который был бы полезен во всех сценариях сразу на многих платформах. На последнем этапе эксперимента мы решили опробовать все наилучшие варианты, полученные для разных конфигураций и сравнить их производительность друг с другом. Например, порядок " EHPDGAJCBNKOLFIMQ " является наилучшим для конфигурации Бенчмарки/windows/x64/, и мы хотели оценить, хорошо ли он подойдёт для конфигурации Библиотеки/Linux/arm64. То же самое — для конфигурации " PEHDCGAIJNLKOBFMQ " (оптимальный вариант для конфигурации Библиотеки/windows/x64) и т.д.

В следующей таблице показаны обобщённые данные по PerfScore , которые мы получили так: брали вариант, признанный наилучшим для одной из конфигураций, и применяли его с другими конфигурациями. В каждой строке указана конфигурация, а также соответствующий оптимальный порядок, предложенный генетическим алгоритмом. В столбцах представлены баллы PerfScore , полученные нами при применении выявленного оптимального порядка к конфигурации, указанной в заголовке столбца.

Конфигурация	Оптимальный порядок	Linux-x64 Benchmarks	windows-x64 Benchmarks	windows-arm64 Benchmarks
windows-x64 Benchmarks	EHPDGAJCBNKOLFIMQ	-83.496405	-36.540712	-19.09969
windows-x64 Libraries	PEHDCGAIJNLKOBFMQ	-85.572973	-35.853492	-19.07247
windows-x86 Benchmarks	EHDCFPGJBIALNOKMQ	-101.903471	-19.844343	-41.041839
windows-x86 Libraries	APCDEGHNIOFJKLBMQ	-26.907257	-0.284718	-30.144657

linux-x64 Benchmarks	HGIDJNLCPOBKA EFMQ	-96.966704	-9.29483	-50.215283
linux-x64 Libraries	HDGAECNIPLBOF KJMQ	-97.682606	-13.882952	-51.929281
linux-arm64 Libraries	HECDBFGIANLOK JMPQ	-97.259922	-11.159774	-54.424627
	Max PerfScore	EHDCFPGJBIAL NOKMQ	EHPDGAJCBN KOLFIMQ	HECDBFGIANLC JMPQ

В последней строке вышеприведённой таблицы указан такой наилучший порядок для конфигурации (из данного столбца), который не входит в число оптимальных вариантов упорядочивания для всех конфигураций. В следующей таблице приведены первый по качеству и второй по качеству варианты порядка для каждой отдельной конфигурации.

Конфигурация	1-й лучший	2-й лучший
windows-x64 Benchmarks	EHPDGAJCBNKOLFIMQ	PEHDCGAIJNLKOBFMQ
windows-x64 Libraries	PEHDCGAIJNLKOBFMQ	EHPDGAJCBNKOLFIMQ
windows-x86 Benchmarks	EHDCFPGJBIALNOKMQ	PEHDCGAIJNLKOBFMQ
windows-x86 Libraries	APCDEGHNIOFJKLBMQ	EHDCFPGJBIALNOKMQ
windows-arm64 Benchmarks	HECDBFGIANLOKJMPQ	HDGAECNIPLBOFKJMQ
windows-arm64 Libraries	HECDBFGIANLOKJMPQ	EHDCFPGJBIALNOKMQ

Если проследить закономерность, характерную для столбца «первый лучший», то увидим, что последовательность Е и Н располагается ближе к началу. Это значит, что, в целом, мы выигрываем, если в качестве одного из первых критериев эвристики обзаводимся `OWN_PREFERENCE` (регистром, который считается предпочтительным в заданном интервале) или `CALLEE_CALLER` (регистрами вызывающей и вызываемой стороны). Далее отметим, что в большинстве вариантов упорядочивания также популярны С и D, соответствующие `THIS_ASSIGNED` (уже присвоен актуальному интервалу) и `COVERS` (захватывает время жизни интервала). В самом начале также идёт эвристика Р, одна из тех, что предназначены для работы с занятыми регистрами. Она соответствует `PREV_REG_OPT` (предыдущая ссылка на присвоенный в настоящее время интервал была опциональной).

Притом, что такие варианты упорядочивания дают хорошие `PerfScore`, наблюдаются и случаи просадок, проявляющихся при использовании других методов. Большинство таких просадок можно отнести к одной или нескольким из следующих категорий:

1. В фазе разрешения алгоритма LSRA есть серьёзные нерешённые проблемы, акцентированные в документе [dotnet/runtime #47194](#). Когда для всех блоков будут найдены ходы, снимающие эти проблемы, их нужно пересмотреть и выяснить, есть ли среди них такие, которые можно сократить в порядке оптимизации. У некоторых методов показатель `PerfScore` падал, так как мы добавляли слишком много таких разрешающих операций на стыках блоков.
2. При всей гибкости, доступной при компоновке различных порядков следования регистров, алгоритм LSRA обладает ограниченной информацией о методе и о том участке кода, для

которого он выделяет регистр. Например, на этапе выделения алгоритм не знает, нужно ли выделить место для выполнения кода и, соответственно, приберечь лишние регистры для использования в этом коде. То есть, ещё до LSRA нужно предусмотреть фазу, на которой эта информация консолидируется в отдельной структуре данных, которую LSRA затем может использовать в ходе выбора регистров.

3. В ходе экспериментов мы выявили и другие легкодоступные оптимизации, касающиеся LSRA при работе с эвристиками переупорядочивания регистров. Например, если переменная определяется всего один раз, то её можно выгрузить в стек именно в той точке, где она была определена, и впоследствии в пределах метода её выгружать не придётся. Это было достигнуто в [dotnet/runtime #54345](#).

Заключение

Выделение регистров — сложная тема. Минимальное изменение алгоритма может сильнее всего сказаться на сгенерированном коде. В этом посте были исследованы различные идеи по поиску оптимальной эвристики для упорядочивания выбора регистров. Оптимальный порядок удалось найти при помощи генетического алгоритма. Также мы обнаружили некоторое сходство в порядке следования эвристик, дававшее выигрыш производительности в большинстве протестированных нами конфигураций. Однако, несмотря на многочисленные улучшения, также во многих методах из всех конфигураций наблюдаются просадки производительности. Мы обнаружили, что есть и другое поле для улучшений, которое нужно проработать до того, как переходить к переупорядочиванию эвристик. Некоторые из проблем затронуты в [\[RyuJIT\]\[LSRA\]](#). Таким образом, на момент подготовки статьи мы решили не менять никаких эвристик, а сначала устранить другие слабые места имеющейся реализации LSRA. Затем можно будет вернуться к этому эксперименту и воспользоваться найденными при этой работе приёмами, инструментами и знаниями об эвристиках переупорядочивания выбора регистров. Далее можно было бы добавить автонастройку порядка выбора регистров, основываясь на разнообразных факторах — например, сколько параметров у метода, есть ли в нём циклы, обрабатываются ли в нём исключения. Возможности безграничны, а время в дефиците, так что нужно стремиться выбирать лучшее!

Теги: [исследования](#), [оптимизация](#), [процессор](#), [алгоритмы](#), [arm](#), [intel](#)

Хэбы: [Программирование](#), [Алгоритмы](#), [C#](#), [.NET](#), [Процессоры](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



32K+

Охват за 30 дней

363

Карма

[@Sivchenko_translate](#)

Пользователь

Подписаться



Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для бэкендеров

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

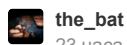
Перейти в поток Бэкенд

Комментарии 1

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K

+96

30

27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой

11 мин

26K

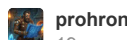
Обзор

Перевод

+71

45

82



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

5 мин

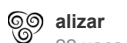
9.3K

Кейс

+51

9

9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

Простой

7 мин

12K

Обзор

+37

59

18



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

🕒 5 мин

👁 8.7K

Обзор



+27



10



0



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плитусом

🔗 Средний

🕒 14 мин

👁 9.9K

Мнение



+26



33



11



mariklolik

18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

🔗 Средний

🕒 7 мин

👁 7.4K

Кейс



+23



23



6



nick_dyuba

22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

🕒 5 мин

👁 7.1K

Recovery Mode



+22



8



5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

🕒 11 мин

👁 7K



+17



9



7



infgotoinf

10 часов назад

Perl — зря забытый язык программирования?

🕒 11 мин

👁 8.5K

Из песочницы



+16



13



8

Тысяча нюансов: как управлять производством лекарств

Турбо

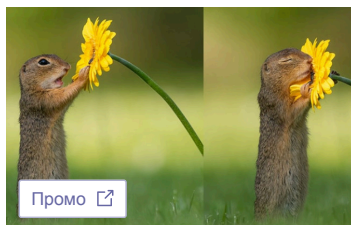
Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Турбо

Удалёнка в фарме: большие данные и никакой магии



Промо

Запахло весной и скидками в Промокодусе



Турбо

Собери облачный пазл и выиграй призы

ВОПРОСЫ И ОТВЕТЫ

Как реализовать поиск слов в файлах?

Алгоритмы · Простой · 3 ответа

Как распознавать полосу для робота на шахматной доске?

Алгоритмы · Простой · 1 ответ

Какие данные берет функция для генерации случайного числа?

Алгоритмы · Простой · 4 ответа

Проблема со связкой ЦАП + DMA в циклическом режиме на gd32f303zet6. Как убрать 0 между циклами?

ARM · Средний · 1 ответ

Почему не решает задачу?

C++ · Простой · 1 ответ

[Больше вопросов на Хабр Q&A](#)

ЧИТАЮТ СЕЙЧАС

Электроинструмент становится хуже, и это делается намеренно

 26K  82

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 13K  14

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 6.9K  15

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

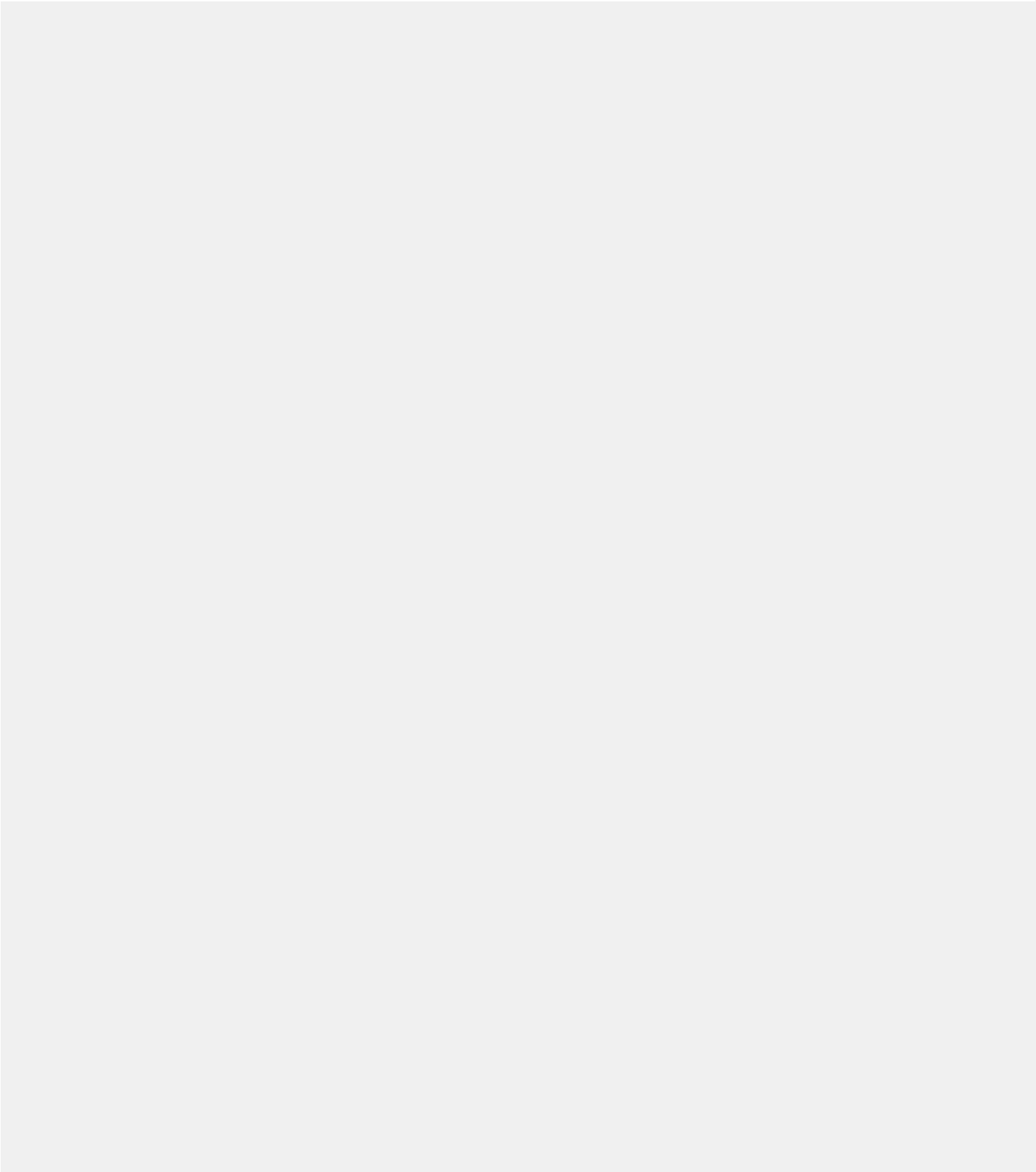
 5.4K  11

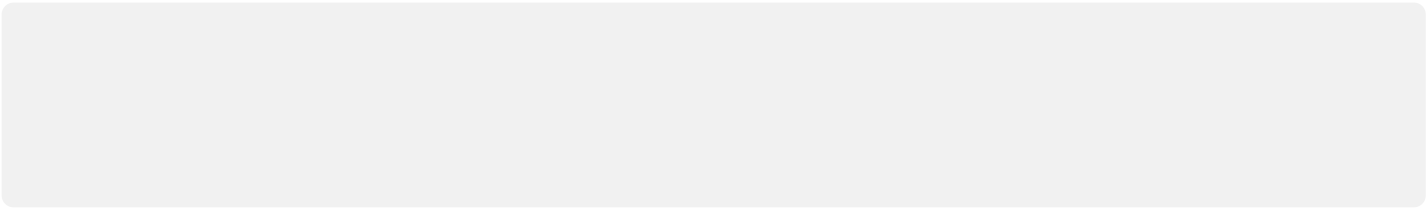
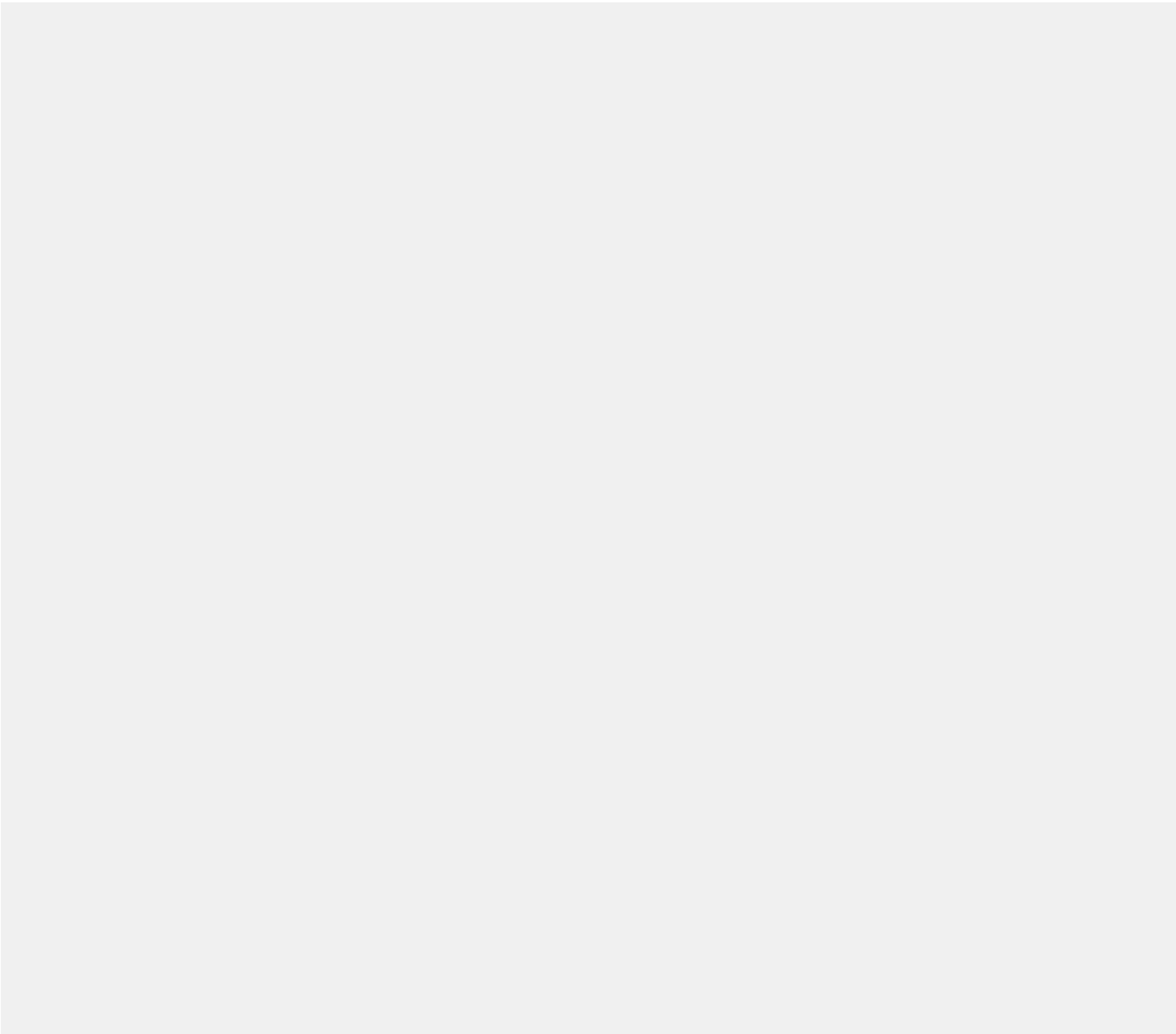
+185% за 13 часов: как Kіmі K2.6 переписала 8-летний движок

 14К  10 

Тысяча нюансов: как управлять производством лекарств

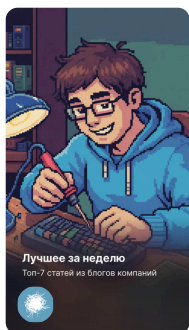
Турбо







ИСТОРИИ



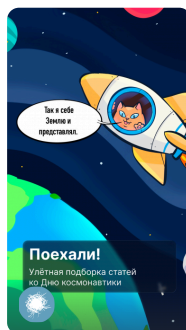
Годнота из блогов компаний



Только 10% решат эту головоломку



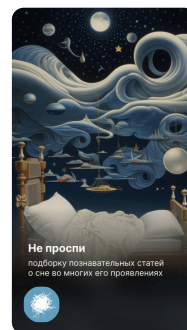
Хочу 450К



Вперёд к звёздам

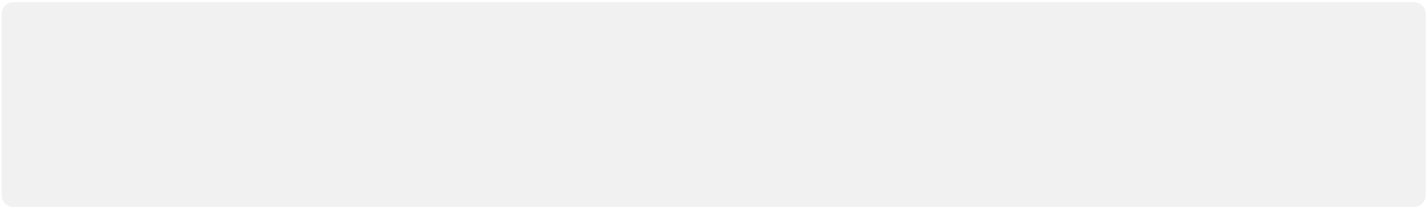
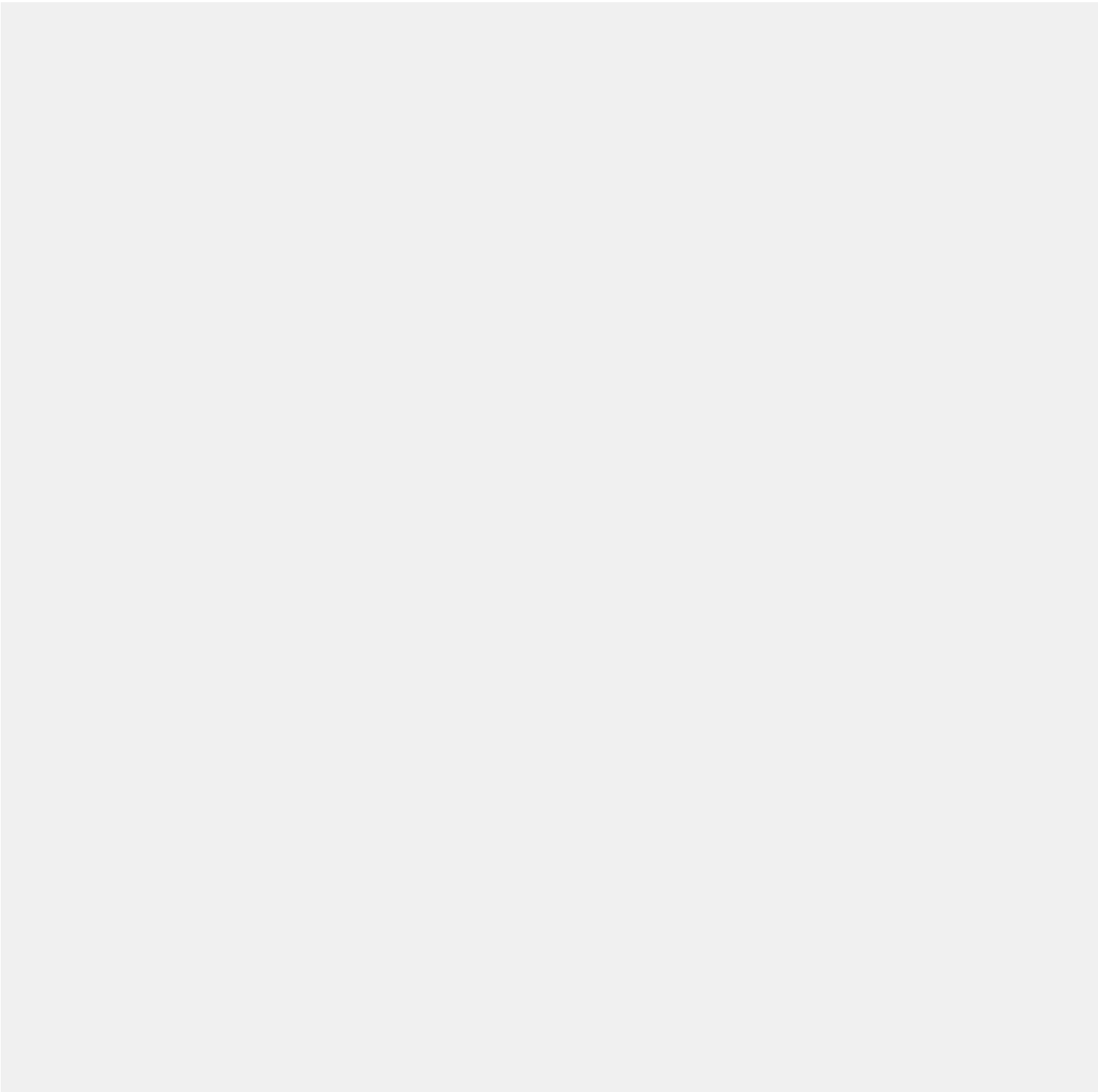


Там на неведомых дорожках...



Самое время поспать







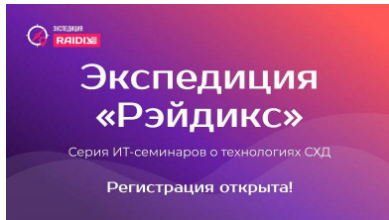
2 марта – 10 августа

PROBA Awards — международная коммуникационная премия, учрежденная в 2000 году

Санкт-Петербург

Маркетинг

[Больше событий в календаре](#)



3 марта – 9 июня

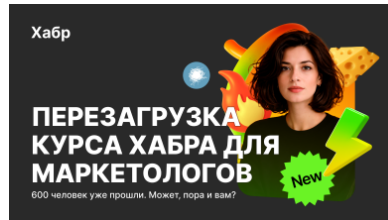
Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск •
Краснодар • Минск • Калининград •
Ростов-на-Дону

Разработка

Другое

[Больше событий в календаре](#)



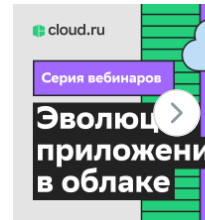
9 марта – 1 июня

Всё, что нужно, чтобы стартовать продвижение бренда на Хабре

Онлайн

Маркетинг

[Больше событий в календаре](#)



19 марта – 28 апреля

Серия вебинаров «Эволюция приложений в облаке»

Онлайн

Разработка

[Больше событий в календаре](#)

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr